

Forensics Git 0

 **Security Warning:** Work in an isolated environment (virtual machine or container). Never open unknown files on your primary machine.

Challenge Description

Can you find the flag in this disk image?

Hints

- How can you extract the directory from the disk image?
-

Tools Used

- `7z` – Extract compressed files
 - `file` – Determine file type
 - `strings` – Extract printable character sequences
 - `exiftool` – Read metadata
 - `fdisk` – View partition tables
 - `dd` – Extract specific partitions
 - `mount` – Mount a filesystem as a loopback device
 - `tree` – Display directory structure
 - `git` – Examine repository history and commits
 - `grep` – Filter output
-

Complete Solution

Step 1: Download and Extract

Download the disk image named `disk.img.gz` from the challenge page.

Extract the compressed disk image:

```
7z x disk.img.gz
```

This decompresses the gzipped file and creates the raw disk image `disk.img`.

Step 2: Basic File Inspection

Verify the file type and understand its structure:

```
file disk.img
```

Expected output:

```
disk.img: DOS/MBR boot sector; partition 1 : ID=0x83, active, start-CHS (0x2,0,33),
end-CHS (0x263,8,56), startsector 2048, 614400 sectors; partition 2 : ID=0x82, start-
CHS (0x263,8,57), end-CHS (0x3ff,15,63), startsector 616448, 524288 sectors; partition
3 : ID=0x83, start-CHS (0x3ff,15,63), end-CHS (0x3ff,15,63), startsector 1140736,
956416 sectors
```

The output shows a **DOS/MBR boot sector** with **three partitions**:

- Partition 1: Linux (ID=0x83) – Boot partition
 - Partition 2: Linux swap (ID=0x82)
 - Partition 3: Linux (ID=0x83) – Data partition
-

Step 3: Search for Visible Text

Disk images often contain strings embedded directly in the data. Extract all readable character sequences and search for the flag:

```
strings disk.img | grep -i "picoCTF"
```

Output:

The picoCTF flag format is 'picoCTF{' where there is some leetspeak phrase in between the curly braces

This confirms the flag format but doesn't reveal the flag itself. We need to dig deeper.

Step 4: Inspect Metadata

Examine the disk image metadata:

```
exiftool disk.img
```

Expected output:

```
ExifTool Version Number      : 13.50
File Name                    : disk.img
Directory                    : .
File Size                     : 1074 MB
File Modification Date/Time   : 2025:11:19 10:53:43+02:00
File Access Date/Time        : 2026:05:14 13:18:40+02:00
File Inode Change Date/Time   : 2026:05:14 13:18:29+02:00
File Permissions              : -rw-r--r--
Error                        : Unknown file type
```

Nothing suspicious here.

Step 5: View Partitions

Use `fdisk` to see the partition structure in detail:

```
fdisk -l disk.img
```

Expected output:

```
Disk disk.img: 1 GiB, 1073741824 bytes, 2097152 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
```

Disk identifier: 0x610b63c2

Device	Boot	Start	End	Sectors	Size	Id	Type
disk.img1	*	2048	616447	614400	300M	83	Linux
disk.img2		616448	1140735	524288	256M	82	Linux swap / Solaris
disk.img3		1140736	2097151	956416	467M	83	Linux

The third partition (disk.img3) is the largest and most likely to contain user data. The first two are likely boot and swap partitions.

Step 6: Extract the Linux Partition

We will extract the third partition using `dd`. The parameters are:

- `skip=1140736` : number of sectors to skip (start of the partition)
- `count=956416` : number of sectors to copy (size of the partition)

```
dd if=disk.img of=linux_partition.dd bs=512 skip=1140736 count=956416 status=progress
```

Output:

```
469463552 bytes (469 MB, 448 MiB) copied, 21 s, 22.4 MB/s
956416+0 records in
956416+0 records out
489684992 bytes (490 MB, 467 MiB) copied, 22.5248 s, 21.7 MB/s
```

Step 7: Analyze the Extracted Partition

Check the type of the extracted partition:

```
file linux_partition.dd
```

Output:

```
linux_partition.dd: Linux rev 1.0 ext4 filesystem data, UUID=7a00e9da-98f8-4f0f-b257-95edf422d902 (extents) (64bit) (large files) (huge files)
```

Excellent! It's an **ext4 filesystem**.

Examine the metadata:

```
exiftool linux_partition.dd
```

Output:

```
ExifTool Version Number      : 13.50
File Name                    : linux_partition.dd
Directory                   : .
File Size                    : 490 MB
File Modification Date/Time  : 2026:05:14 18:06:46+02:00
File Access Date/Time       : 2026:05:14 18:07:14+02:00
File Inode Change Date/Time  : 2026:05:14 18:06:46+02:00
File Permissions             : -rw-r--r--
Error                        : First 1024 bytes of file is binary zeros
```

Step 8: Mount the Extracted Partition

Create a mount point and mount the partition as a loopback device:

```
sudo mkdir /mnt/git0
sudo mount -o loop linux_partition.dd /mnt/git0
```

Note: The `-o loop` option allows Linux to treat a regular file as a block device.

Step 9: Explore the Mounted Image

List the contents of the root directory:

```
ls -alh /mnt/git0
```

Output:

```
total 45K
drwxr-xr-x 22 root root 1.0K Nov 19 10:38 .
drwxr-xr-x 13 root root 4.0K May 14 17:22 ..
drwxr-xr-x  2 root root 4.0K Nov 19 10:39 bin
drwxr-xr-x  2 root root 1.0K Nov 11 2025 boot
```

```
drwxr-xr-x  2 root root 1.0K Nov 11  2025 dev
drwxr-xr-x 30 root root 3.0K Nov 19 10:39 etc
drwxr-xr-x  3 root root 1.0K Nov 11  2025 home
drwxr-xr-x 10 root root 1.0K Nov 11  2025 lib
drwx----- 2 root root  12K Nov 11  2025 lost+found
drwxr-xr-x  5 root root 1.0K Nov 11  2025 media
...
```

After searching, let's look inside the `home` directory for any interesting files (like a `.git` repository):

```
tree /mnt/git0/home -alh
```

Output:

```
[1.0K] /mnt/git0/home
├─ [1.0K] ctf-player
│   └─ [1.0K] Code
│       └─ [1.0K] secrets
│           └─ [1.0K] .git
│               ├── [ 62] COMMIT_EDITMSG
│               ├── [ 23] HEAD
│               ├── [1.0K] branches
│               ├── [ 92] config
│               ├── [ 73] description
│               ├── [1.0K] hooks
│               ├── [ 478] applypatch-msg.sample
│               └── [ 896] commit-msg.sample
...
```

Key finding: A `.git` repository exists at `/home/ctf-player/Code/secrets/.git` ! This likely contains the flag.

Step 10: Examine the Git Repository

Navigate to the `.git` directory and use Git commands to explore the repository history:

```
cd /mnt/git0/home/ctf-player/Code/secrets/.git
```

First, check the Git log:

```
git log
```

Output:

```
commit 327681bb38cf467cec328eec9707b240e3e74ced (HEAD -> master)
Author: ctf-player <ctf-player@example.com>
Date:   Wed Nov 19 08:49:27 2025 +0000
```

Wrap this phrase in the flag format: <REDACTED>

Now, view the commit details:

```
git show
```

Output:

```
commit 327681bb38cf467cec328eec9707b240e3e74ced (HEAD -> master)
Author: ctf-player <ctf-player@example.com>
Date:   Wed Nov 19 08:49:27 2025 +0000
```

Wrap this phrase in the flag format: <REDACTED>

```
diff --git a/note.txt b/note.txt
new file mode 100644
index 00000000..46064ac
--- /dev/null
+++ b/note.txt
@@ -0,0 +1 @@
+The picoCTF flag format is 'picoCTF{ }' where there is some leetspeak phrase in
between the curly braces
```

The commit message contains the flag phrase! The actual flag is redacted here, but in the real challenge, it will be displayed.

Alternative Git Commands

Here are some useful Git commands for exploring repositories in forensic contexts:

Command	Purpose
<code>git log --oneline</code>	Show commits in a compact format (one line per commit)
<code>git log --stat</code>	Show commit statistics (files changed, insertions/deletions)
<code>git log -p</code>	Show full diff for each commit
<code>git show <commit-hash></code>	Show details of a specific commit
<code>git diff HEAD~1 HEAD</code>	Show changes between the last two commits
<code>git reflog</code>	Show reference log (useful for recovering "lost" commits)
<code>git branch -a</code>	List all branches (local and remote)
<code>git tag</code>	List all tags
<code>git stash list</code>	Show stashed changes
<code>git fsck --lost-found</code>	Recover dangling commits/blobs that are no longer referenced
<code>git grep "picoCTF"</code>	Search for a pattern across all tracked files
<code>git blame note.txt</code>	Show who last modified each line of a file
<code>git whatchanged</code>	Show a log of what changed in the repository

 Final Result

picoCTF{<REDACTED>}

Note: The actual flag is redacted here. In the real challenge, the `git show` command will reveal the complete flag phrase inside the commit message.

 Key Concepts

Concept	Description
Disk Image (DD)	A raw sector-by-sector copy of a storage device.
MBR Partition Table	A structure that divides a disk into multiple partitions (max 4 primary partitions).
Partition Types	<code>0x83</code> = Linux filesystem, <code>0x82</code> = Linux swap, <code>0x82</code> = Linux swap / Solaris.
Partition Extraction with <code>dd</code>	Using <code>skip</code> and <code>count</code> to isolate a specific partition from a disk image.
ext4 Filesystem	The native Linux filesystem, commonly used for system and data partitions.
Loopback Mount	Mounting a file as a block device using <code>mount -o loop</code> .
Git Repository	A version control directory (<code>.git</code>) that stores the complete history of a project.
Git Commit Messages	Can contain important information like flag phrases.
Forensic Git Analysis	Examining <code>.git</code> directories to recover deleted files, commit history, and hidden information.

 Pro Tip

When dealing with disk images, always check for `.git` directories, `.svn` , `.hg` , or other version control artifacts. They frequently contain the entire development history, including files that may have been deleted or modified, making them a goldmine for forensic analysis.